Managing State and Front-End Development

A lot of the decisions about program structure in web applications are about how to manage *state*, which is the data that controls the interactions between the user and the application. Managing state includes both the location of that data itself and the location of the logic that manipulates that state. Once you have the program structure set, you also have to worry about packaging, or how to convert your developer-friendly code to browser-friendly code.

The Structure of Web Applications

One of the main questions we'll be dealing with is how to structure your web application so as to best manage your state. The goal is to avoid having multiple sources of truth, both by avoiding duplicating data as well as avoiding writing the same logic on both the client and the server side. We also want to make the program as easy as possible to understand and change.

A consistent problem in web development is that as far as the browser and HTTP server are concerned, the interaction for each page view is "stateless." Being stateless means that each interaction is completely self-contained. As far as the web server is concerned, each request has no relation to or memory of previous requests.

This lack of state is quite useful if you are a web server, because it makes your life much easier not to have to keep track of any state. If you are a user of the web, however, the lack of state is annoying because the web server never remembers anything about you. Web applications depend on maintaining your state to remember who you are and what you are doing, so developers have created different solutions to manage the state of a user's interaction with a web server.

Almost since the beginning of the web, a technical solution to this problem has been *cookies*. A cookie is a small amount of data—which often consists of a random string of characters—generated by the server and managed by the browser. The cookie allows the browser to identify itself, and an application